

Assignment 2

Learning Outcomes

This assignment is intended to develop and assess the following unit learning outcomes:

- ✓ **LO1.** Iteratively apply object-oriented design principles to design small to medium-size software systems, using standard software engineering notations, namely UML class diagrams and UML interaction diagrams.
- ✓ **LO2.** Describe the quality of object-oriented software designs, both in terms of meeting user requirements and the effective application of object-oriented design concepts and principles.
- ✓ **LO3.** Apply object-oriented programming constructs, such as abstraction, information hiding, inheritance, and polymorphism, to implement object-oriented designs using a programming language (namely, Java).
- ✓ **LO4.** Apply effective programming strategies to refactor and debug object-oriented implementations systematically and efficiently using available programming language tools.
- ✓ **LO5.** Apply principles of software engineering practice to create object-oriented systems with peers using tools including integrated development environments (IDEs), UML drawing tools, and version control systems.

To demonstrate your ability, you will be expected to:

- read and understand UML design documentation for an existing Java system
- propose a design for additional functionality for this system
- create UML class diagrams to document your design using a UML drawing tool such as [diagrams.net](#), [UMLet](#) or [plantuml](#) – you are free to choose which one
- write a design rationale evaluating your proposed design and outlining some alternatives
- implement the features of the system that you designed
- use an integrated development environment to do so
- use git to manage your team's files and documents

The marking scheme for this assignment will reflect these expectations

Learning Materials

The base code will be automatically available in your group's repository (Gitlab).

Repeat this mantra: Design, write code, test, fix design, fix code, repeat



Note: You **must NOT follow** demo apps' design decisions; they only show how to use the engine, **NOT** how

to design a proper system with object-oriented principles.

Introduction



Heavy penalty will apply for editing (add, modify, delete) any classes in the game engine as it breaks the purpose of the whole learning experience.

Before working on assignment 2, you and your team members need to agree on which design diagram, rationale and implementation will be used for the starting point for assignment 2.

You need to push the diagram, rationale and implementation to the new group repository.



IMPORTANT: You may change your design if you find that you need to. It is normal for the design to evolve alongside the codebase as the developers' understanding of the requirements improves. If your design changes, you should document the changes. This includes your design rationale as well as your diagrams.

REQ1: The Ancient Woods

After fighting their way out of the burial ground, the player somehow managed to escape successfully in one piece. After looking within their surroundings, the player finds themselves in a new area, the Ancient Woods.

```
".....+++.....+++++++.....~~~.....~~~",
"+.....+++.....+++++++.....~~~.....~~~",
"++.....#####.....++++.....~~~.....~~~",
"++.....#_____#.....~~~.....~~~",
"+.....#_____#.....~~~.....~~~",
".....###_###.....~.....~~~.....~~~",
".....~.+++~.....~~~.....~~~",
".....~.....~+++++.....~~~.....~~~",
".....~~~.....++++.....~~~~~.....~~~",
".....~~~~,~~~~.....~.....~~~~~.....~",
"++++.....~~~~~.....~~~.....~~~~~.....~",
"++++.....~~~~~.....~~~.....~~~~~.....~"
```

A sample Ancient Woods map



The Ancient Woods must be implemented as a new map (separate from the abandoned village and the burial ground maps). The player still needs to unlock a gate and go through it to be able to move to the new map.



Make sure to add another Gate in the Ancient Wood map to go back to the Burial Ground map. You can assume that this is a separate gate that the player also needs to unlock.



If you don't change your implementation of moving between maps from assignment 1, you don't need to include this feature in the class diagram since it has been documented in the assignment 1 diagram. However, if you made any changes to this functionality, you need to include them here.

After arriving at the new area, the player sees several seemingly empty huts (represented as "h"). Watching them from afar, the player sees a humanoid creature coming out of the huts. This creature is another enemy that the player has to fight against, known as the "Forest Keeper", displayed as "8" in the game. The huts can spawn this enemy with a 15% chance at each turn. The "Forest Keeper" has 125 hitpoints and can attack the player with their limbs, dealing 25 damage with 75% accuracy.

As the player attempts to run away to hide in nearby bushes (represented as "m"), they get ambushed by a "Red Wolf", displayed as "r". The bushes can spawn this enemy with a 30% chance at every turn. The "Red Wolf" has 25 hit points and can attack the player by biting them, dealing 15 damage with 80% accuracy.



Similar to the previous enemies from Assignment 1, if the player is not within the surrounding of the enemies,

they will wander around. However, when the player is nearby (within the surrounding of the enemies), they will attack the player. An enemy cannot attack another enemy.

The enemies cannot enter a floor, represented by the display character “_”, so that the player can run back to safety if their health is low. Furthermore, both the player and the enemies cannot enter walls. They, however, can walk around freely in the Void*, on puddles and dirt.

* with consequences

All enemies in the Ancient Woods can **follow** the player if the player is within the surrounding of the enemies. For simplicity, you can assume that once an enemy follows the player, they will keep following until either the player is unconscious or the enemy is unconscious.

Once defeated, the “Forest Keeper” has a 20% chance to drop a healing vial. While the “Red Wolf” has a 10% chance of dropping a healing vial.

REQ2: Deeper into the Woods

After defeating the inhabitants of the ancient woods, the player notices that the enemies dropped “Runes” (represented by “\$”), which is the currency used in the world of “Designborne”.



“Runes” must be picked up and used/consumed by the player to increase their wallet balance.



Don't forget to print out the number of runes that is being held by the player at a particular turn.

The “Forest Keeper” drops 50 runes, while the “Red Wolf” drops 25 runes. Going back to the enemies from Assignment 1, the “Wandering Undead” drops 50 runes, while the “Hollow Soldier” drops 100 runes.



In this game, you can assume that all enemies drop “Runes” once they have been defeated by the player, but the amount they drop varies from one enemy to another. The enemy will not drop runes if they are unconscious due to environmental hazards, such as the Void.

Going deeper into the woods, the player decides to drink from the nearby puddle of water (represented by “~”). In the game, since the player does not carry around a container for the water, they should be able to consume the water from the ground directly without having to add anything to their inventory. Drinking from the puddle of water heals the player by 1 point and restores the player's stamina by 1% of their maximum stamina.

Edit (18/09): The player can only drink water from the puddle that they are standing on.



The player should also be able to drink water from the puddles of water found in the previous maps (the burial ground and the abandoned village).

Nearby, the player notices a bloodberry (represented by “*”) lying on the ground, which the player can pick up and consume. After consuming the bloodberry, the player's maximum health is increased by 5 points permanently.



You can add Bloodberry manually with the following code:

```
gameMap.at(x, y).addItem(new Bloodberry());
```

REQ3: The Isolated Traveller

Deep within the forest, the player comes across an old building where a suspicious traveller lives (represented by " "). Approaching the traveller inside the building allows the player to purchase items from the traveller's inventory. The player can also sell items from their inventory to the traveller.



You can assume that the traveller does not move around (that is, they will only stay at one spot in the building).



A reminder: a building is simply several squares of floor (`. `) and dirt (`. `), surrounded by walls (`#`).

The traveller offers the following items from their inventory that can be purchased by the player:

- Healing vials for 100 runes each (when the player purchases a healing vial, there is a 25% chance that the traveller asks the player to pay 50% more. If the player's balance is less than what the traveller asks for, the purchase fails).
- Refreshing flasks for 75 runes each (when the player purchases a refreshing flask, there is a 10% chance that the traveller gives a 20% discount to the player)
- Broadwords for 250 runes each (when the player purchases a broadsword, there is a 5% chance that the traveller takes the player's runes without giving the broadsword)



For simplicity, you can assume that the traveller has an unlimited supply of items that they can sell to the player.

The player can sell all items in their inventory to the traveller for the following price:

- Healing vials for 35 runes each (when the player sells a healing vial, there is a 10% chance that the traveller pays 2x the original price)
- Refreshing flasks for 25 runes each (when the player sells a refreshing flask, there is a 50% chance that the traveller takes the item without paying the player)
- Broadwords for 100 runes each
- Bloodberries can be sold for 10 runes each
- Old keys cannot be sold

Edit (22/09): For simplicity, you can assume that the same item will have a fixed selling price across different traders. So, Trader A and Trader B will buy Item J at the same price from the player. Different traders, however, may sell the same item for different prices.



For simplicity, the traveller can buy items from the player regardless of the traveller's wallet balance.

The traveller cannot be attacked by the player.

REQ4: The room at the end of the forest

Before leaving the traveller's building, the traveller offers the player to purchase a new weapon: a Great Knife. This weapon deals 75 damage with 70% attack accuracy. It is represented by the character ">". Also, this weapon has a special skill, "stab and step", which allows the player to attack an enemy and step away to safety within the same turn. This weapon is sold by the traveller for 300 runes each. This special skill consumes 25% of the player's maximum stamina.

When the player purchases a great knife, there is a 5% chance that the traveller will ask the player to pay 3x the original price of the weapon. If the player's balance is less than what the traveller asks for, the purchase fails.

The player can sell the weapon back to the traveller for 175 runes, but there is a 10% chance of the traveller taking the runes from the player instead. If the player has less than 175 runes, the traveller will only take what the player has left (the player's balance will never be negative).



For simplicity, you can choose a new location randomly within the player's surroundings after attacking with the "Stab and step" skill using the great knife. You don't need to make sure that the new location is the exact opposite of the enemy.

At the end of the forest, the player notices another locked gate that can be unlocked by the old key.

```
"~~~~.....+++.....~+++++.....",
"~~~~.....+++.....+++++.....",
"~~~++.....+++.....+++++.....",
"~~~++.....++.....+.....+",
"~~~~~.....+.....~~~++.....++",
"~~~~~.....++++.....~~~+++++.....++",
"~~~~~.....+++++++~.....++++.....++",
"~~~~~.....+++++++~.....+++.....++",
".....++++.....++.....++",
".....+~~.....++",
".....~~~~.....++",
".....~~~++.....+",
".....++++.....+++++.....",
".....++++~.....+++++.....",
".....+++~.....++++.....~",
".....++.....++++.....~~",
".....+++++.....~~",
".....+++++++.....~~",
".....~+++++.....~",
".....~~~+++++.....~..~"
```

A sample Abxervyer battle map



For now, don't add a separate gate in the new map to go back to the Ancient Woods. Refer to REQ 5 as to why.

Inside the room, there are several huts and bushes that the player encountered earlier in the Ancient Forest (see requirement 1), which spawn the "Forest Keeper" enemy and the "Red Wolf" enemy, respectively.



If you don't change your implementation of moving between maps from assignment 1, you don't need to include this feature in the class diagram since it has been documented in the assignment 1 diagram. However, if you made any changes to this functionality, you need to include them here.

When entering the room, the player notices yet another weapon: the Giant Hammer (represented by the character "P"). This weapon deals 160 damage with 90% attack accuracy. It has a special skill: "Great Slam". This lets the player slam a specific enemy, dealing 100% of the weapon damage to only that enemy while also dealing 50% of the weapon's original damage to any actor within the enemy's surroundings, including the player themselves. This special skill consumes 5% of the player's maximum stamina.

This weapon can be sold to the traveller for 250 runes, but it cannot be bought from the traveller.

REQ5: Abxervyer, The Forest Watcher

In the middle of the room, There is a tall figure standing, known as “Abxervyer, the Forest Watcher” (represented by “Y”). This triggers the first boss encounter that the player has to fight against in the game. The boss has 2000 hit points and can attack the player with its limbs, dealing 80 damage with 25% accuracy.



As with the previous enemies, if the player is not within the surrounding of the boss, they will wander around. However, when the player is nearby (within the surrounding of the enemies), they will attack the player. The boss cannot attack another enemy.

The boss cannot enter a floor, represented by the display character “_”, so the player can run back to safety if their health is low. Furthermore, both the player and the enemy cannot enter walls. They, however, can walk around freely on puddles and dirt.

All enemies, including the boss, in the Ancient Woods can follow the player if the player is within the surroundings of the enemies. For simplicity, you can assume that once an enemy follows the player, they will keep following until either the player is unconscious or the enemy is unconscious.

Note that the boss will not get hurt if they walk around in the Void.

The boss has a special ability: they can control the weather in the forest.

When the weather is sunny:

- The huts spawn the “Forest Keeper” enemy at 2 times the original spawning rate (i.e. 30% instead of 15%)
- The “Red Wolf” enemy deals 3 times the original damage when attacking the player (i.e. 45 damage points instead of 15 damage points)
- Bushes’ spawning rate returns to normal

When the weather is rainy:

- The bushes spawn the “Red Wolf” enemy at 1.5 times the original spawning rate (i.e. 45% instead of 30%)
- The “Forest Keeper” enemy is healed by 10 points.
- Huts’ spawning rate and red wolf damage return to normal

The weather alternates every three turns. This repeats until the boss is defeated by the player.



For simplicity, you can assume that the boss can control the weather as long as the player has not defeated the boss, even when the player is outside of the boss room.



The boss room must be implemented as a new map (separate from the other maps). The player still needs to unlock a gate and go through it to be able to move to the new map.

Once the boss is defeated, the location where the boss last stood turns into a Gate, which can be unlocked by the player. If the player enters this gate, it will bring the player back to the Ancient Wood. The boss also drops 5000 runes when defeated. Additionally, you should print a message when the boss is defeated.

Submission Instructions



Within this class, you are welcome to use foundation models (ChatGPT, GPT, DALL-E, Stable Diffusion, Midjourney, GitHub Copilot, and anything after) in a totally unrestricted fashion, for any purpose, at no penalty. However, you should note that all large language models still have a tendency to make up incorrect facts and fake citations, code generation models have a tendency to produce inaccurate outputs, and image generation models can occasionally come up with highly offensive products. You will be responsible for any inaccurate, biased, offensive, or otherwise unethical content you submit regardless of whether it originally comes from you or a foundation model. If you use a foundation model, its contribution must be acknowledged in the submission (in the form of a separate file titled `FoundationModelsUsed.txt`); you will be penalised for using a foundation model without acknowledgement.

Having said all these disclaimers, the university's policy on plagiarism still applies to any uncited or improperly cited use of work by other human beings, or submission of work by other human beings as your own. Moreover, missing contribution logs/GIT commits and/or failing any handover interview will be treated as a potential breach of academic integrity which will be further investigated.



We will mark your assignment based on the latest commit in the `main` branch in your GitLab repository by the due date, **not** in the `master` branch.



Heavy penalty will apply for editing (add, modify, delete) any classes in the game engine as it breaks the purpose of the whole learning.



A Reminder:

Before working on assignment 2, you and your team members need to agree on which design diagram, rationale and implementation will be used for the starting point for assignment 2.

You need to push the diagram, rationale and implementation to the new group repository.

You are expected to produce the following artefacts in Assignment 2:

1. Implementation (the code) of the system.
2. Contribution Log (mandatory, useful for markers) - see <https://edstem.org/au/courses/12156/lessons/37194/slides/259339>
3. Revised (based on feedback received on Assignment 1 or if you consider changes were needed)
+ New UML design documents, which include
 - 5 UML class diagrams (1 for each requirement, similar to Assignment 1)
 - 5 Interaction diagrams (1 for each requirement: sequence diagrams or communication diagrams are equally acceptable)
4. Design rationale, which should include (5 of them, 1 for each requirement)
 - Justification for any changes made to the design proposed for Assignment 1
 - Justification for the new requirements



For class diagrams, please make sure to distinguish between existing classes and new classes with different colours. Additionally, you should colour revised classes and newly added classes differently.

Additionally, you will be asked to complete the following forms:

- Peer Assessment Form (mandatory, useful for markers and for you to reflect on your group dynamics)
- Feedback Request Form (to be completed in order to upload your files to Moodle).

As mentioned above, you can (and should) **update** the artefacts previously marked in Assignment 1 since the **code must align with the UML diagrams**.

We will mark your Assignment 2 on the state of the `main` branch of your Monash GitLab repository by the due date at your local time (see above). If you've done any work in other branches, make sure you merge it into main before the due time.



The FIT2099 Assignment Rules apply to this assignment. Please read this document and make sure you comply, especially as regards the Work Breakdown Agreement.

You do not need to create a new copy of your work for Assignment 2. Git lets us easily roll back to any previous version of your repository, so keeping a separate "Assignment 1" folder only creates unnecessary clutter. If you like, you can [add a tag](#) to your final Assignment 1 commit before starting on Assignment 2, so you can find that version easily.

You may update your design if you find that you need to. We expect consistency between your design documents and implementation. Regarding the contributions document, please use the prepared tab to separate the contribution from the previous assignment. We will take your contribution logs into account when marking.

It is highly recommended that you explicitly Indicate in the **design rationale** what changed in the design between Assignment 1 and Assignment 2. Explaining the changes would help you organise your ideas to explain how the design changed during your handover interview. A quick summary of changes with some explanation regarding why you applied such changes would be really valuable.

Contribution Log

We require you to have a document to know how you plan to divide the work between team members and simultaneously capture individual contributions. We have prepared a template you can copy to your university's Google Drive and share with your team. Please follow the detailed instructions [here](#).



Since your markers need to access this information, make sure that you create a share link and put it in the README of your repository. Failure to do so will result in 0 mark.

In this matter, everyone must contribute a **FAIR amount of code AND documentation**. It means you cannot work only on the code or only on documentation. In other words, your team **MUST NOT**

split the work by artefacts: a person working on a "class diagram", another on a diagram alone, and another working on design rationale alone. We will give a heavy penalty to your team as a whole.

You will meet up with your team ideally once a week for about 1 hour for team discussion and another 1 or 2 hours for integration and resolving conflicts (if any).

Peer Assessment Form

You should also complete the peer assessment form, which you can find here:

<https://forms.gle/mTK8WM6LZ345y8aQ7>, to help us understand and evaluate your (and other team members') contributions to the assignment better.

About Late Submissions

It is all team members' responsibility to ensure that the correct versions of the documentation and code are present in the repository by the due date and time.

Marking Rubric

Assignment 2 Rubric

Prerequisite for marking the whole assignment: Design documents (UML diagrams and design rationale) and implementation need to be submitted for the assignment to go through the marking process. Missing any of these will result in 0 marks.

Overview:

In this assignment, the basic requirements (REQs 1-4) and the challenging requirement will be marked separately (REQ 5).

Basic requirements are worth 80% of the marks (REQs1-4)

Total: 26 points

- Feature implementation completeness (6 points)
- Implementation quality: design principles (8 points)
- Design rationale (4 marks)
- Integration with the existing system (2 points)
- UML syntax and clarity (2 points)
- Alignment and design consistency (2 points)
- Style & Javadoc (1 point)
- Format and directory structure (1 point)

The challenging requirement is worth 20% of the marks (REQ 5)

~~**Prerequisite for marking:** evidence that all the team members contributed to this requirement (with at the very least 1 meaningful commit) is needed for the assignment to go through the marking process. Missing this evidence will result in 0 marks for this requirement.~~

Total: 24 points

- Feature implementation completeness (6 points)
- Implementation quality: design principles (8 points)
- Design rationale (4 points)
- Integration with the existing system (2 points)
- UML syntax and clarity (2 points)
- Alignment and design consistency (2 points)

Detailed rubric:

Feature implementation completeness (6 points)

6 marks - The system runs and perfectly meets **all functional expectations** as per the relevant requirement(s) with no runtime errors. All required classes and relationships are implemented, and the program's behaviour aligns perfectly with the specification.

5 marks - The system runs and meets **all functional expectations (with some minor errors or punctual omissions)** as per the relevant requirement(s). All necessary classes and relationships are included, and the program's behaviour largely matches the specification, except for **one or two minor unexpected behaviours**. No runtime errors occur.

4 marks - The system runs and **partially meets all functional expectations, displaying several minor functional errors or omissions** as per the relevant requirement(s). Despite this, the majority of necessary classes and relationships are included, and **only a few minor unexpected behaviours occur**. No runtime errors are present.

3 marks - The system runs and **all functional expectations were addressed to some extent**, (with one or two major functional errors) as per the relevant requirement(s). Most important classes and relationships are included but **at least one or two major unexpected behaviors** are observed. A major unexpected behaviour is that which affects considerably the completeness of at least one requirement. No runtime errors occur.

2 marks - The system runs but **several functional expectations were not addressed** as per the relevant requirement(s) (e.g. the notion of Behaviour is not included even though it is a part of the requirement). Alternatively, runtime errors occur during the execution of the system but they can be easily fixed.

1 mark - The system runs but **addresses only some functional expectations and shows major omissions** as per the relevant requirement(s) (important classes or relationships are missing). Alternatively, runtime errors occur during the execution of the system without a clear idea of the cause or cannot be easily fixed.

0 marks - The system runs but it **poorly addresses or doesn't address the functional expectations** as per the relevant requirement(s). Alternatively, the system might not run at all.

Implementation quality: design principles (8 points)

This item applies across all functional expectations as per the relevant requirement(s)

8 marks - The implementation of all functional expectations as per the relevant requirement(s) **flawlessly adheres to good design principles and concepts** (e.g., DRY, SOLID principles and connascence), making the design easy to extend and maintain. Any punctual violations are **convincingly** justified in the design rationale (e.g., using singleton to implement a feature). All relevant requirements have been addressed.

7 marks - The implementation follows good design principles **nearly perfectly** making the design is easy to extend and maintain (if some punctual principles are violated, the trade-off is **somewhat** justified in the design rationale. All relevant requirements have been addressed.

6 marks - The implementation involves **one or two minor violations of design principles** (could be easily fixed) across all functional expectations as per the relevant requirement(s) (e.g., some attributes are not set to private without any justification). All relevant requirements have been addressed.

5 marks - The implementation involves **minor violations of design principles in multiple places** (could still be easily fixed) across all functional expectations as per the relevant requirement(s). All relevant requirements have been addressed.

4 marks - The implementation involves **one or two non-severe violations of design principles that could be implemented in a better way** (no trade-offs are convincingly provided in the design rationale) across all functional expectations as per the relevant requirement(s) (e.g. some code repetitions are found in the implementation that would require some refactoring to be fixed). All relevant requirements have been addressed.

3 marks - The implementation involves **non-severe violations of design principles in multiple places** (no trade-offs are convincingly provided in the design rationale). All relevant requirements have been addressed.

2 marks - The implementation involves **one or two severe violations** of design principles that could be implemented in a better way across all functional expectations as per the relevant requirement(s), e.g. violating the SOLID principles. Fixing them would require substantial refactoring. Alternatively, not all relevant requirements have been attempted.

1 mark - The design/implementation can be considered **hacky or various instances of procedural programming are found**. For example, the implementation uses downcasting and 'instanceof' in various cases without a convincing justification. Alternatively, it can also be the case that abstraction is not used, making the design **difficult to extend and maintain**. Alternatively, only some relevant requirements have been attempted.

0 marks - The implementation mainly follows a non-OO paradigm; or the UML class diagram(s) or the implementation is missing. Alternatively, most relevant requirements have not been attempted.

Design rationale (4 points)

4 marks - The design rationale includes a description of **what has been done and why**, focusing on the principles and concepts taught in the unit such as DRY, SOLID principles, connascence etc., and not in terms of game design. The rationale discusses the **advantages and disadvantages** of the design (pros and cons), **the reasons** for choosing the current design, and **ways in which it can be easily extended** (e.g. my design achieves OCP because if a new character is added in the future ...). All relevant requirements have been addressed.

3 marks - The rationale describes **what has been done and why**, based on the principles and

concepts taught in the unit. It also includes **some discussion** of the pros and cons of the design and the reasons for the current design choice but **lacks examples of future extensibility**. All relevant requirements have been addressed.

2 marks - The rationale describes **what has been done and why**, aligning with the principles and concepts taught in the unit. However, it **lacks a discussion on the pros and cons of the design and/or examples of future extensibility**. All relevant requirements have been addressed.

1 mark - The rationale **primarily describes what has been done** without a thorough explanation of the decision-making reasons. For instance, it may mention design principles and concepts without providing convincing explanations, discussion of pros and cons, or examples of how the system can be extended. Alternatively, not all relevant requirements have been attempted.

0 marks - The design rationale is either very challenging to read, is missing, or the submitted work omits more than one relevant requirement.

Integration with the existing system (2 points)

This item applies across all relevant requirements.

2 marks - The implementation **effectively uses the engine classes** (e.g. the submitted work demonstrates that the students understand the difference between actions and behaviours). All relevant requirements have been addressed.

1 mark - The implementation **does not use some engine classes as intended** (e.g. behaviours are created for some actions that do not need it, such as actions performed by the player) or includes **custom classes for functionality that could be implemented with the engine class** (e.g. created a custom ground class instead of using the ground class given in the engine package). Most relevant requirements have been addressed.

0 marks - The engine has been modified in a minor or significant way OR the UML class diagram OR the implementation is missing

UML syntax and clarity (2 points)

This item applies across all relevant requirements.

2 marks - Relevant (static and/or dynamic) diagrams are **perfect in terms of syntax**, with no missing multiplicities, correct arrowheads for all relationships, and appropriate usage of realisation for classes implementing interfaces, generalisation for classes or interfaces inheriting others, etc. Diagram formatting is **consistent and clear**. All requested diagrams have been submitted.

1 mark - Diagram(s) contain **some minor syntax errors**, such as missing multiplicities for several associations, inappropriate use of generalisation for classes implementing interfaces, realisation for classes extending others, or classes extending multiple classes, etc. Nonetheless, the design can still be understood by the TA with a little effort. Alternatively, there are several **inconsistencies across diagrams**. All necessary diagrams have been submitted.

0 marks - Diagram(s) are significantly hard to understand or show major inconsistencies in formatting and clarity, OR more than one required diagram is missing, OR they do not resemble a UML diagram as required.

Alignment and design consistency (2 points) - incl. Design rationale, code and UML diagrams

2 marks - The design rationale and UML diagrams are in **perfect alignment** with each other and with the code implementation across all functional expectations, as per the relevant requirement(s). All relevant requirements have been addressed or attempted.

1 mark - There are **some small inconsistencies** (that can be easily fixed) between the design documents and the implementation, but all relevant requirements have been addressed or attempted.

0 marks - There are **major inconsistencies** (which would necessitate significant changes for correction) or **numerous smaller discrepancies** distributed throughout the design documents and the implementation. Alternatively, more than one required diagram or implemented requirement is missing, or some submitted design documents do not resemble the required UML diagram format.

Style & Javadoc (1 point)

Applicable to basic requirements (1-4) only

1 mark - The code is properly documented with Javadoc across all functional expectations as per the relevant requirement(s), including class-level and method-level documentation. The Google Java Style guide is followed properly (e.g. package names are written in lowercase, attributes and variables names are written in lowerCamelCase, class names are written in UpperCamelCase, etc.). All relevant requirements have been addressed or attempted.

0.5 marks - Some classes and methods are missing Javadoc documentation OR some classes do not follow the Google Java style guide

0 marks - Most classes and methods are missing Javadoc documentation OR The Google Java style guide is not followed.

Project formatting and directory structure (1 point)

Applicable to basic requirements (1-4) only

1 mark - If the directory structure suggested in the specification has been followed. All the documents are easily found where expected.

0 marks - The suggested directory structure was not followed and some files may be hard to find.

Individual Contribution (100%) & Handover Interview

(compulsory)

We will, in principle, assume all team members equally contributed to the assignment. Yet, this individual contribution component will be used to weigh the marks awarded at a group level (see above) for cases where individual students did not contribute to the group task fairly (see details below) or did not show evidence that they understand their own work during the handover interview (see details under Handover Interview, below).

If you contributed to the group task and demonstrate that you understand your own work you should get (100%) the full marks awarded at a group level as per the rubric above.

Individual Contribution (100%) Only for A2 and A3

100-80% - The student has made substantial contributions to the submitted work, evident by numerous **commits** (over 10) with meaningful commit comments (note: default comments from web UI are not considered). For full marks, there should be **at least one commit per week leading up to the submission deadline, and multiple contribution log entries** (over 10 entries) with comprehensible explanations for an external reader (e.g. the TA).

Additionally, the student's contributions reflect a reasonable understanding of OOP principles, whether in programming or designing UML diagrams. The student's input spans **both design and code**.

Additionally, other team members have given positive feedback or there are no complaints regarding the student's contributions (as **per peer assessment**).

70-40% - The student has made some contributions to the submitted work, evident by **some commits** (between 5 and 9) with meaningful commit comments OR the student made more than 10 commits but **all of them on the same week of the submission deadline** or there may be **commits without any comments**.

Alternatively, there are **some contribution log entries** (between 5 and 9) with comprehensible explanations for an external reader (e.g. the TA).

Alternatively, the student's input focused **only on the design or the implementation**.

Alternatively, there are **some complaints** regarding the student's contributions (as per peer assessment).

30-20% - Peer assessment, contributions logs, and git logs show there are **insufficient contributions from an individual**; the student received **a negative review** from other team members which usually happened due to last-minute work, difficult communication outside of labs, or any other kind of difficulties (evidence are collective from peer assessment, emails, contribution logs, and git logs). The student committed to git, with **at least four commits** and meaningful comments OR there are various commits that are not meaningful and that do not explain the purpose. **Some contribution log entries** (at least 4 entries) with meaningful explanations that can

be understandable by a new team member (e.g. the TA).

10-0%% - There is no or little evidence of contribution to the team from Git logs or contribution logs. If applicable, the student only writes comments (inline or Javadoc) without any appropriate programming contribution. Clear evidence of academic misconduct of an individual would be marked as zero. Note: submitting files as attachments to the release notes would be marked as zero.

Handover Interview (compulsory)

COMPLETION If the student can answer all (or at least 2) questions during the handover interview satisfactorily. The responses need to demonstrate that the student understands the various parts of the assignment both parts created by the student (as per the logged commits) and at least some awareness of those parts created by other team members (for Assignments A2 and A3).

NOT COMPLETED If two or more questions are not responded to adequately and sensibly. The remaining question(s) is/are partly responded to, but it is unclear whether the student understands their own work.



IMPORTANT: Failing to have meaningful commits (i.e. showing that the task was progressively completed) and/or failing the handover interview would automatically flag this as a potential case of plagiarism, it will be further investigated using a similarity check software, and **zero marks would be awarded**.



IMPORTANT: Any inquiry (e.g. potential conflict within a team) should be submitted via the emails below (not your AdminTA, Lecturer nor CE). Emails sent in other ways will not be processed in time.

FIT2099.Clayton-x@monash.edu if you are based in Clayton

FIT2099.Malaysia-x@monash.edu if you are based in Malaysia

Assignment 2 Q&A Session

Direct link: <https://monash.au.panopto.com/Panopto/Pages/Viewer.aspx?id=4cae4509-f714-47e6-a05c-b085006f9d4b>

Meeting chat:

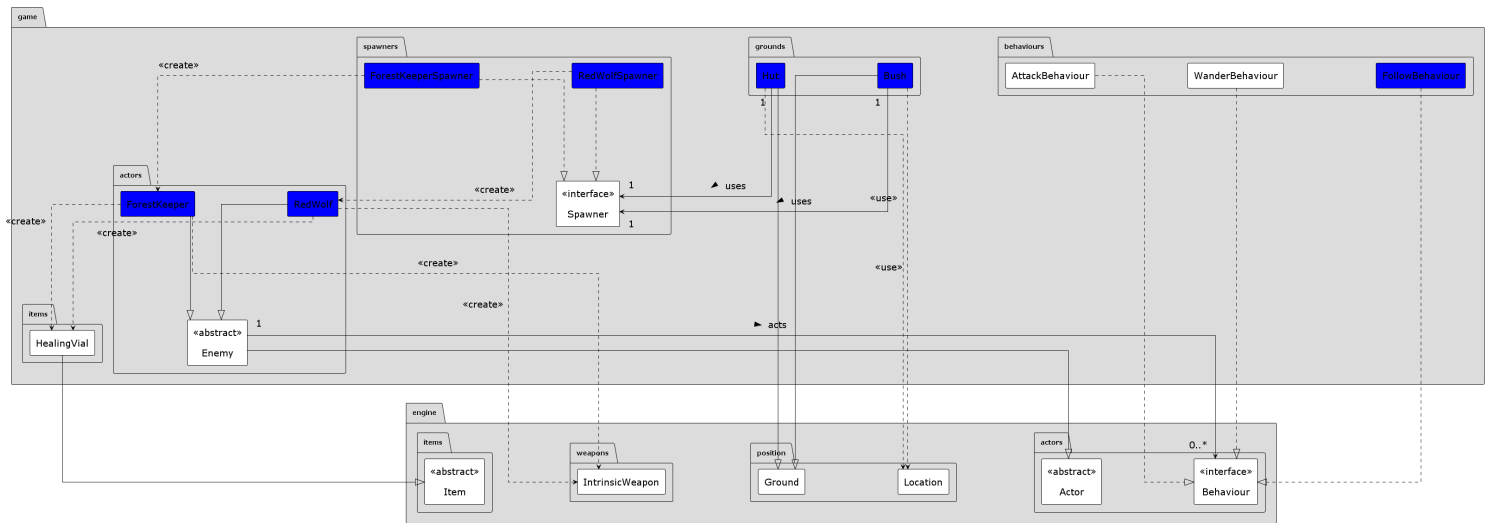


GMT20230922-050621_RecordingnewChat.txt

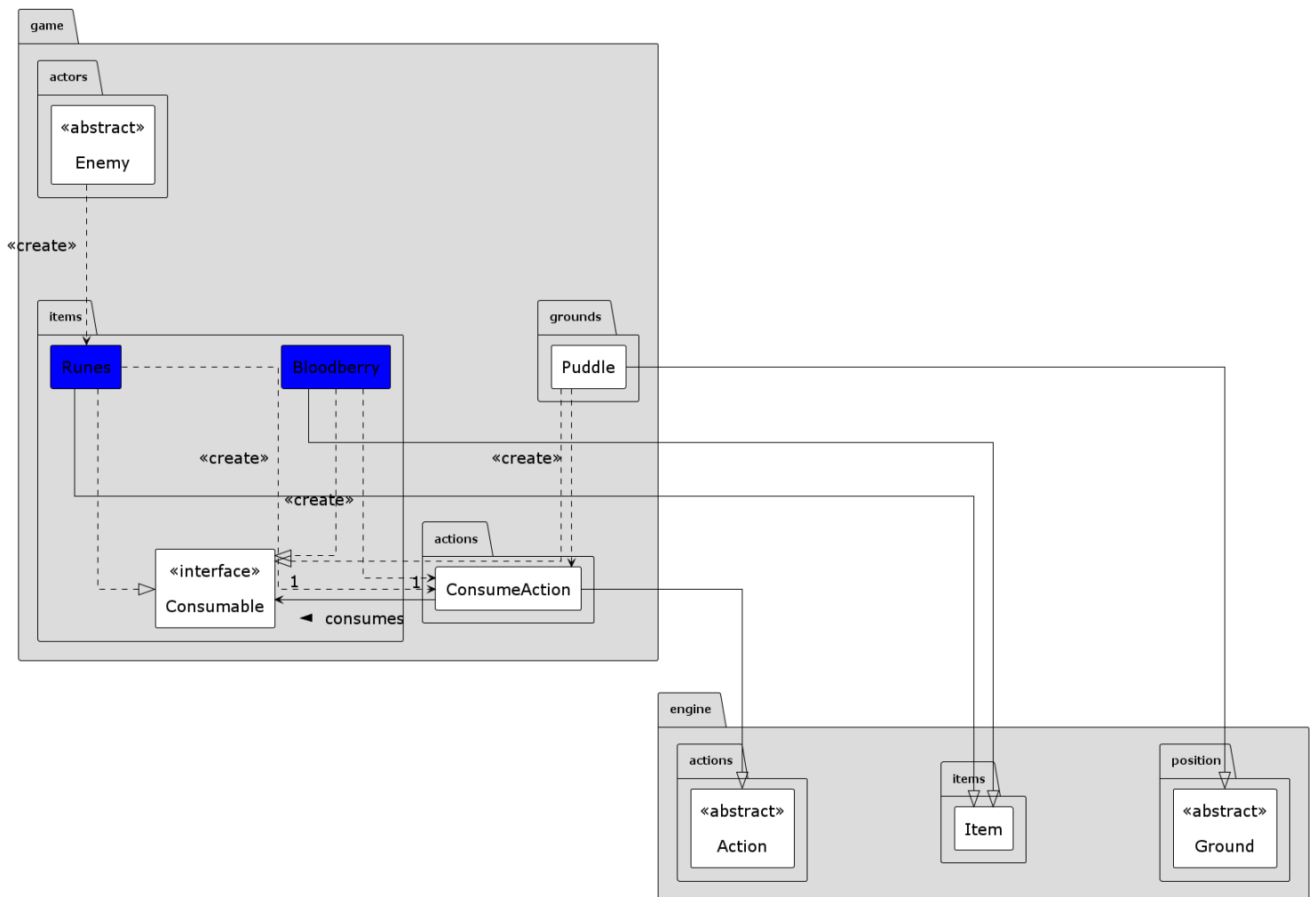


Assignment 2 Sample UML class diagrams

REQ1



REQ2



REQ3

